Project: Evolutionary Search for a Domain of Your Choice

Bryn Mawr CS 367: Software Analysis

Groups: 2–3 students

Overview

In lectures 14-16 we saw how a genetic algorithm (GA) searches for exact or approximate solutions to search and optimization problems. EvoSuite uses one to generate test suites: a chromosome is a test case, fitness is coverage, and selection, crossover, and mutation evolve the population over many generations.

In this project you will build your own evolutionary algorithm for a domain of your choice, then run experiments to see how its parameters affect the search and how it compares to random search.

Timeline

All deadlines are at 11:59 pm.

Checkpoint 1: Sunday, November 15. Submit a 1–2 page document covering Parts 1 and 2: your domain and your answers to the Part 2 design questions.

Checkpoint 2: Sunday, November 29. Submit a 2–3 page document (including plots) with initial experiments: results from your baseline configuration and at least one experiment from Part 4.1.

Final report: Friday, December 18. See Deliverable below.

Part 1: Choose a Domain

Pick a problem where:

1. **A candidate solution can be represented as a chromosome.** A list, string, grid, or small structure works well.
2. **Fitness is computable.** Given any individual, you can automatically score how good it is.
3. **The search space is large.** Brute-force enumeration should be impractical. If you can try every solution, you don't need a GA.

Some ideas to get you started. You are welcome to pick something else.

Paths *(an ordered list of stops or moves)*
A route through national parks; a walk across the city that avoids surveillance cameras; a wheelchair-accessible route across campus; a night walk home along well-lit streets.

Subsets *(in or out for each item)*
A grocery list under a budget that meets nutrition needs; a fantasy sports team under a salary cap; which bills a legislator should co-sponsor to build the widest coalition; what to pack for a backpacking trip.

Schedules *(a time slot for each event)*
A final exam timetable; a semester course schedule that meets a student's requirements; shifts for a coffee shop or hospital.

Layouts *(positions in space)*
A keyboard layout; turbines in a wind farm; locations for new bike-share stations.

Strategies *(numbers that control behavior)*
Blackjack strategy; a betting strategy in a simulated game; when to buy and sell in a simulated market.

Part 2: Design Your GA

Before writing code, answer each question below for your domain. Your answers become the Design section of your report.

Chromosome. What is one candidate solution, and how will you represent it?

Initial population. How will you generate random individuals? Are they always valid?

Fitness. How will you score an individual? Is higher or lower better?

Selection. How will parents be chosen? (You will implement both roulette wheel and rank selection.)

Crossover. How will two parents combine into offspring?

Mutation. What small random changes can you make to an individual?

Termination. When will the search stop?

Think about invalid individuals. If crossover or mutation can produce something that breaks your constraints (a knapsack over the weight limit, a route that visits a city twice), decide whether to repair it, reject it, or penalize it in the fitness function.

Part 3: Implement

Implement your GA following the general structure of Algorithm 1, adapted from [1]. You may use any language, but you must write the GA yourself. Do not use a GA library such as DEAP or Jenetics.

Your implementation must expose the following as **parameters** (command-line flags or a config file, not hard-coded constants):

- **Population size**
- **Selection method:** implement at least **roulette wheel** and **rank selection**.
- **Crossover probability**

- **Mutation probability**
- **Termination criteria:** max generations and/or timeout
- **Random seed**, so runs are reproducible

Your program should log, for every generation, the best fitness and the mean fitness of the population.

Algorithm 1 Evolutionary Algorithm

```
Initialize population P with random solutions
Evaluate fitness of each individual in P
while search budget not exhausted do
    Select individuals from P based on fitness
    Apply crossover to produce offspring
    Apply mutation to offspring
    Evaluate fitness of offspring
    Add offspring to P
end while
```

Part 4: Experiments

GAs have no convergence guarantee, and a single run tells you very little. For every configuration, run **at least 10 trials with different seeds** and report the mean and spread (standard deviation or a box plot). For each experiment, include a **fitness-over-generations plot** and a table of final results.

4.1 Parameter study

Start from a **baseline configuration** you choose and justify. Then change **one parameter at a time**:

1. **Population size:** at least 3 values (e.g., 10, 50, 200). Keep the total number of fitness evaluations comparable, or explain why you didn't.
2. **Selection method:** roulette wheel vs. rank selection. Does one very fit individual dominate under roulette wheel on your problem? What happens if your fitness can be negative?
3. **Mutation probability:** at least 3 values, including 0. Does the search get stuck in local optima without mutation? What happens when mutation is very high?
4. **One more of your choice:** crossover probability, crossover method, elitism, or termination criteria.

4.2 Comparison to random search

Implement random search: repeatedly generate a random individual (using the same generator as your initial population) and keep the best one seen. Give random search the **same budget** as your GA, measured in fitness evaluations, not wall-clock time.

Compare your best GA configuration to random search. Does evolution actually help on your problem? If not, why not? (A GA that loses to random search is a fine result if you can explain it.)

Deliverable

Submit a **report** (5-7 pages) with the following sections:

- **Problem:** your domain and why it is a search / optimization problem
- **Design:** your answers to the Part 2 questions, with justification
- **Experiments:** setup, plots, and results for Parts 4.1–4.3
- **Discussion:** Which parameter mattered most? What surprised you? Based on your comparisons, would you use a GA for this problem?

References

[1] Elizabeth Dinella. On the Effectiveness of Modular Testing in EvoSuite. In *Proceedings of the ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis (SOAP)*, 2026. arXiv:2604.27112.